



Obsidian

AUDITS

Tenor Security Review

Auditors

Oxjuaan

OxSpearmin

July 15, 2026

Introduction

Obsidian Audits

Obsidian audits is a team of top-ranked security researchers, with a publicly proven track record, specialising in DeFi protocols across EVM chains and Solana.

The team has achieved 10+ top-2 placements in audit competitions, placing 1st in competitions for Wormhole, Pump.fun, Yearn Finance, and many more.

Find out more: obsidianaudits.com

Audit Overview

Tenor is a fixed-rate lending protocol on Morpho Midnight that lets borrowers and lenders automatically renew their loans as they approach maturity: users set their rate, duration, and gating preferences once, and keepers roll the position into a fresh fixed-rate loan at a matching offer.

Tenor engaged Obsidian Audits to perform a security review of the smart contracts in the `tenor-morpho-v2-contracts-2` repo. The initial diff review was conducted from June 15, 2026 to June 18, 2026, and a post-fix diff review was conducted from July 2, 2026 to July 4, 2026.

Scope

Files in scope

Repo	Files in scope
tenor-morpho-v2-contracts-2 Scope: diff review Starting commit: f0c736973b44995b96233cc2acb c302b53527fc2 Ending commit: 0d9ab57f98d1501777400177bd49 e06cb43106a3	src/**/*.sol Excluding: src/external src/periphery/clamps
Scope: post-fix diff review (repo migrated between commits) Starting commit: tenor-morpho-v2-contracts-2 0d9ab57f98d1501777400177bd49 e06cb43106a3 Ending commit: tenor-contracts 7d259111e6b327e69be9972c7a03 c4c23db97516	src/**/*.sol Excluding: src/external src/periphery/clamps

Summary of Findings

Severity Breakdown

A total of 5 issues were identified and categorized based on severity:

- 1 Low severity
- 4 Informational

Findings Overview

Initial Diff Review

f0c7369 → 0d9ab57

ID	Title	Severity	Status
I-01	Incomplete credit guard in LendMidnightRenewalCallback	Informational	Fixed
I-02	onSell callbacks don't enforce `receiver == address(this)`	Informational	Fixed

Post-Fix Diff Review

0d9ab57 → 7d25911 (repo migrated to tenor-contracts)

ID	Title	Severity	Status
L-01	`assets == max` silently skips a callback funded repay in `midnightRepay`	Low	Fixed
I-03	Redundant approval in `midnightRepay` when a callback funds the repay	Informational	Fixed
I-04	Incomplete test coverage	Informational	Fixed

Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users. Alternatively, breaking a core aspect of the protocol's intended functionality
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - the attack/vulnerability requires minimal or no preconditions, but there is limited or no incentive to exploit it in practice
- **Low** - requires highly unlikely precondition states, or requires a significant attacker capital with little or no incentive.

Findings

[I-01] Incomplete credit guard in LendMidnightRenewalCallback

Initial diff review — `f0c7369` → `0d9ab57`

Description

The `creditAfterUpdate == 0` check intends to surface an explicit, named revert for the no-credit case, rather than letting Midnight's `withdraw` revert via underflow. But it only does this for the zero credit case ([LendMidnightRenewalCallback.sol#L52](#)). The rest of the insufficient-credit range (`0 < credit < neededAssets`) still falls through to the `credit -= units` underflow in Midnight's `withdraw`.

Recommendation

Consider adding a separate guard for the insufficient credit range, while keeping the existing zero guard:

```
uint256 neededAssets = buyerAssets + fee;
+ if (neededAssets > creditAfterUpdate) revert
CallbackLib.InsufficientCredit();
```

Remediation: Fix confirmed

[I-02] onSell callbacks don't enforce `receiver == address(this)`

Initial diff review — `f0c7369` → `0d9ab57`

Description

Introduced by [PR #469](#), `onSell` callbacks gained a `receiver` param: the address Midnight transfers the proceeds (`sellerAssets`) to, before the callback is invoked.

Several callbacks ignore `receiver` and spend `sellerAssets` from their own balance. All four document in NatSpec that `receiverIfMakerIsSeller` must be the callback (so `sellerAssets` is transferred to them before `onSell` runs), but none enforce `receiver == address(this)` in code. `receiver` and the callback are independent fields, so `receiver` can be set to an address different to the callback. Example: `MidnightSupplyVaultSharesCallback.onSell` deposits `sellerAssets` into a vault from its own balance; `BorrowBlueToMidnightCallback` / `BorrowMidnightRenewalCallback` / `LendMidnightToVaultCallback` repay/migrate the same way.

Recommendation

This is not exploitable since callbacks are not expected to hold funds, however it's recommended to have an explicit revert if the receiver is not set to the callback:

```
if (receiver != address(this)) revert CallbackLib.InvalidReceiver();
```

This check should be added to the `onSell` function in the following callbacks:

```
BorrowBlueToMidnightCallback, BorrowMidnightRenewalCallback,  
LendMidnightToVaultCallback, MidnightSupplyVaultSharesCallback.
```

Remediation: Fix confirmed

[L-01] `assets == max` silently skips a callback funded repay in `midnightRepay`

Post-fix diff review — `0d9ab57` → `7d25911` (repo migrated to `tenor-contracts`)

Description

When `assets == type(uint256).max`, `MidnightAdapterBase.midnightRepay()` sizes the repay from the adapter's own loan-token balance.

```
} else if (assets == type(uint256).max) {  
    units = IERC20(market.loanToken).balanceOf(address(this));  
}
```

This is only correct when the adapter is the one paying for the repay.

When a `callbackAddr` is supplied, Midnight sets the callback as the payer and pulls the loan tokens from it, while the adapter itself holds none. The balance reads `0`, `units` resolves to `0`, and the `if (units == 0) return;` on the next line returns early. `Midnight.repay` is never called, the callback is never invoked, and the debt is never reduced, yet the call still succeeds silently.

Consider a bundle including `midnightRepay` followed by a `midnightWithdrawCollateral`. The user intends to clear their debt (funded by the vault-share redeem callback) and then withdraw some of the freed collateral.

Because the repay silently resolves to `0` and is skipped, the debt is never reduced, but the collateral withdrawal still runs. Some of the collateral is pulled out while the full debt remains, leaving the position at a higher LTV than the user intended.

The position cannot be pushed past liquidation, since `Midnight.withdrawCollateral` enforces `isHealthy`, so the withdrawal reverts or is capped there. But within that bound it is left unhealthier than it originally was.

Recommendation

In `MidnightAdapterBase.midnightRepay()`, Consider rejecting `assets == type(uint256).max` when `callbackAddr != address(0)`

```
require(assets != type(uint256).max || callbackAddr == address(0),
InconsistentInput());
```

Remediation: Fixed in PR [#17](#)

[I-03] Redundant approval in `midnightRepay` when a callback funds the repay

Post-fix diff review — `0d9ab57` → `7d25911` (repo migrated to `tenor-contracts`)

Description

Before calling `repay`, `midnightRepay` sets a max loan-token approval from the adapter to Midnight. That approval is only consumed when the adapter is the payer.

When `callbackAddr != 0`, Midnight resolves the payer to the callback and pulls funds from it instead, so the approval call from the adapter is not needed in that case.

Recommendation

Consider only approving Midnight when `callbackAddr == address(0)` (the adapter is the payer).

Remediation: Fixed in PR [#18](#)

[I-04] Incomplete test coverage

Post-fix diff review — `0d9ab57` → `7d25911` (repo migrated to `tenor-contracts`)

Description

Various contracts have branches which are not reached by any tests in the test suite. It's recommended to add tests to cover these cases.

Running `forge coverage ---ir-minimum` fails due a "stack too deep" error, which can be fixed by applying [this diff](#) to `midnight/src/ratifiers/libraries/HashLib.sol`.

After applying the diff locally, `forge coverage --report lcov` can be used to identify the branches which are uncovered by tests.

Remediation: Fixed in PR [#19](#)